UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	**Angga Syahputra (2428240065)**
Kelas	SI5B
Judul project / topik API	Tugas 2 – RESTful API Rute Bus
Tautan repositori	https://github.com/Zamn48/Tugas-2-Automation-Postman
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	22/10/2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

API Rute Bus merupakan layanan RESTful API yang digunakan untuk mengelola informasi rute transportasi bus. API ini menyediakan fitur untuk melihat, menambahkan, memperbarui, dan menghapus data rute bus melalui metode HTTP. Pengembangan API menggunakan Express.js dengan pemisahan fungsi ke dalam beberapa bagian, seperti routes, controllers, dan models, sehingga pengelolaan kode menjadi lebih terstruktur

1.2 Daftar endpoint

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/bus-routes	Tidak	200	Menampilkan seluruh rute bus
2	GET	/bus-routes/:id	Tidak	200	Menmpilkan detail rute berdasarkan ID
3	GET	/bus-routes?kota=Palembang	Tidak	200	Mengambil data rute berdasarkan kota
4	POST	/bus-routes	Tidak	201	Menambahkan rute bus baru
5	PUT	/bus-routes/:id	Tidak	200	Mempernarui informasi rute bus
6	DELETE	/bus-routes/:id	Tidak	204	Menghapus rute bus berdasarkan ID

1.3 Variabel Environment

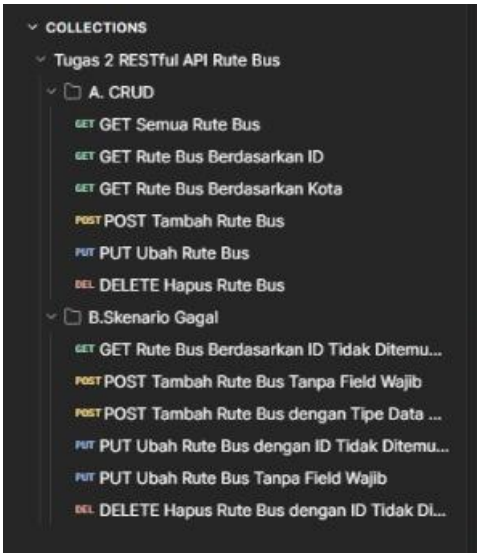
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000/	Alamat server
2	busRoutesId	Kosong pada awal pengujian	Menyimpan ID rute bus dari respons POS

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:
Postman Collection terdiri atas dua folder, yaitu A. CRUD dan B. Skenario Gagal, yang masing-masing berisi enam request. Folder CRUD digunakan untuk menguji fungsi utama API Rute Bus, sedangkan folder Skenario Gagal menguji penanganan ID yang tidak ditemukan dan data yang tidak valid. Seluruh request disusun berdasarkan metode HTTP sesuai fungsinya untuk memudahkan proses pengujian API

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Semua Rute Bus	GET	{{base_url}}/bus-routes	A. CRUD
2	GET Rute Bus Berdasarkan ID	GET	{{base_url}}/bus-routes/1	A. CRUD
3	GET Rute Bus Berdasarkan Kota	GET	{{base_url}}/bus-routes?kota=Palembang	A. CRUD
4	POST Tambah Rute Bus	POST	{{base_url}}/bus-routes	A. CRUD
5	PUT Ubah Rute Bus	PUT	{{base_url}}/bus-routes/{{busRouteId}}	A. CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
6	DELETE hapus Rute Bus	DELETE	{{base_url}}/bus-routes/{{busRouteId}}	A. CRUD
7	GET ID Rute bus tidak Ditemukan	GET	{{base_url}}/bus-routes/99	B. Skenario Gagal
8	POST Data Rute Bus Tidak Lengkap	POST	{{base_url}}/bus-routes	B. Skenario Gagal
9	POST Tarif dengan Tipe Data Salah	POST	{{base_url}}/bus-routes	B. Skenario Gagal
10	PUT Rute Bus Tidak Ditemukan	PUT	{{base_url}}/bus-routes/99	B. Skenario Gagal
11	PUT Data Rute Bus Tidak Lengkap	PUT	{{base_url}}/bus-routes/{{busRouteId}}	B. Skenario Gagal
12	DELETE ID Rute Bus Tidak Ditemukan	DELETE	{{base_url}}/bus-routes/99	B. Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	GET Semua Rute Bus	Status 200	Respons berbentuk array dan memuat minimal tiga data awal
2	GET Rute Bus Berdasarkan ID	Status 200	Memeriksa atribut id, kodeRute, asal, tujuan, kota, dan tarif
3	GET Rute Bus Berdasarkan Kota	Status 200	Respons berbentuk array dan setiap data memiliki kota Palembang
4	POST Tambah Rute Bus	Status 201	Memeriksa status success, kode rute K4, tarif 5000, dan menyimpan ID
5	PUT Ubah Rute Bus	Status 200	Memeriksa data setelah perubahan
6	DELETE Hapus Rute Bus	Sesuai respons API	Memeriksa status penghapusan dan respons sesuai implementasi
7	GET ID Rute Bus Tidak Ditemukan	Status 404	Memeriksa penanganan ID yang tidak ditemukan
8	POST Data Rute Bus Tidak Lengkap	Status 400 jika validasi tersedia	Memeriksa penolakan data yang tidak lengkap
9	POST Tarif dengan Tipe Data Salah	Status 400 jika validasi tersedia	Memeriksa penolakan tipe data tarif yang salah
10	PUT Rute Bus Tidak Ditemukan	Status 404 jika ditangani API	Memeriksa penanganan ID yang tidak ditemukan
11	PUT Data Rute Bus Tidak Lengkap	Status 400 jika validasi tersedia	Memeriksa penolakan data perubahan yang tidak lengkap
12	DELETE ID Rute Bus Tidak Ditemukan	Status 404 jika ditangani API	Memeriksa penanganan penghapusan ID yang tidak ditemukan

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
pm.test("Status code 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Data rute bus berhasil ditampilkan", function () {
    const data = pm.response.json();

    pm.expect(data).to.be.an("array");
    pm.expect(data.length).to.be.at.least(3);
});
```

Penjelasan:
Assertion pertama memeriksa apakah request GET Semua Rute Bus menghasilkan status code 200. Assertion kedua memastikan respons berbentuk array dan memuat minimal tiga data rute bus. Dengan demikian, terdapat dua Test Results untuk memeriksa keberhasilan pengambilan data

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
pm.test("Status code 201", function () {
    pm.response.to.have.status(201);
});

pm.test("Rute bus berhasil ditambahkan", function () {
    const json = pm.response.json();

    pm.expect(json.status).to.eql("success");
    pm.expect(json.data.kodeRute).to.eql("K4");
    pm.expect(json.data.tarif).to.eql(5000);
});

const json = pm.response.json();

if (json.data && json.data.id !== undefined) {
    pm.collectionVariables.set("busRouteId", json.data.id);
}
```

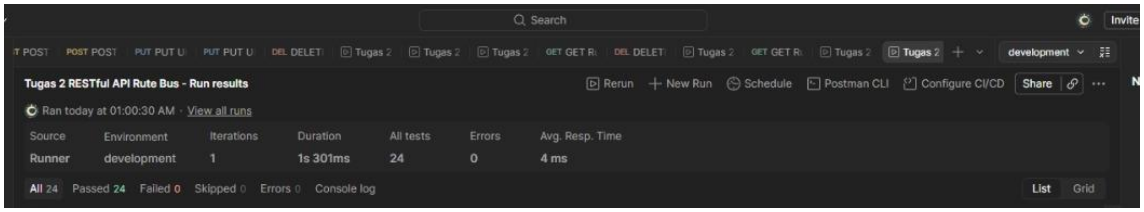
Penjelasan:
Script memeriksa status code 201 dan memastikan rute K4 dengan tarif 5000 berhasil ditambahkan. ID dari respons kemudian disimpan ke Collection Variable busRouteId agar dapat digunakan pada request PUT dan DELETE. Struktur assertion harus sesuai dengan respons POST yang benar-benar dihasilkan API

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:
Berdasarkan hasil Collection Runner, pengujian API Rute Bus memperoleh 24 test passed, 0 failed, dan 0 skipped dalam satu iterasi. Hasil ini menunjukkan bahwa seluruh pengujian berjalan sesuai dengan assertion yang telah dibuat pada 12 request di Postman Collection.

4.2 Rekap angka hasil

Nama Collection	Tugas 2 – RESTful API Rute Bus
Environment yang dipakai	development
Jumlah request dijalankan	12
Jumlah test lulus	24
Jumlah test gagal	0
Tanggal dan jam pengujian	Isi sesuai waktu pengujian sebenarnya

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	POST Tambah Rute Bus	Isi jika terjadi kegagalan	Isi berdasarkan hasil pemeriksaan	Isi perbaikan yang dilakukan

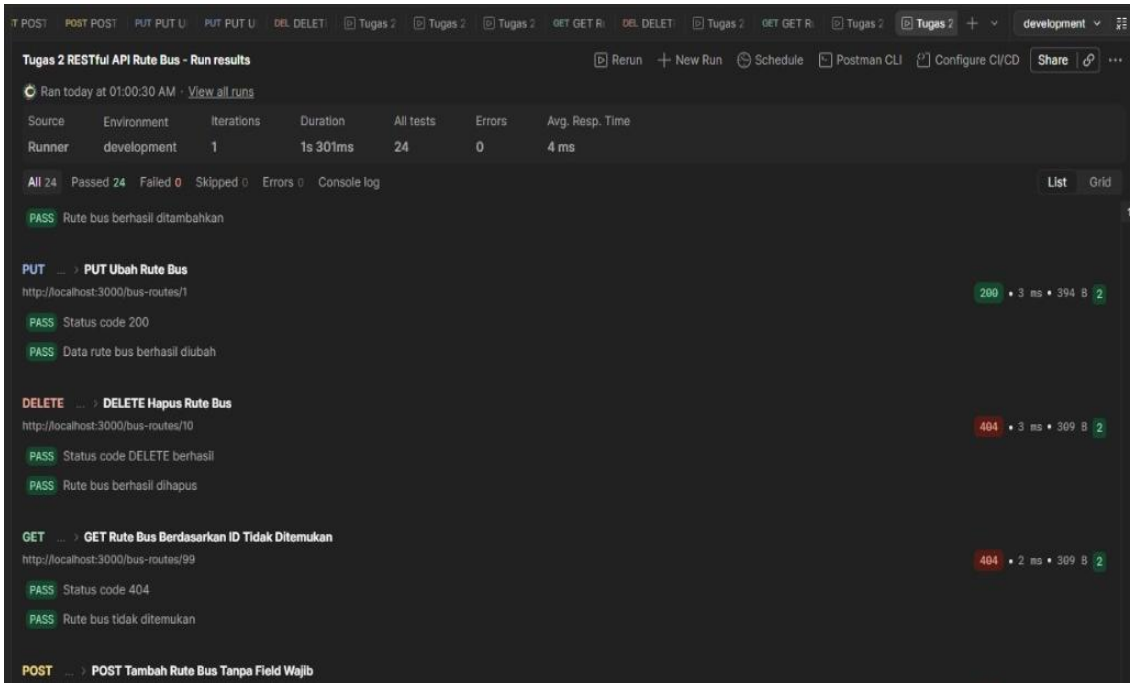
No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
2	PUT Ubah Rute Bus	Isi jika terjadi kegagalan	Isi berdasarkan hasil pemeriksaan	Isi perbaikan yang dilakukan

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.



Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Pengujian RESTful API Rute Bus dilakukan menggunakan fitur Collection Runner pada Postman untuk memastikan bahwa setiap endpoint dan skenario pengujian berjalan sesuai dengan yang diharapkan. Pengujian dilakukan terhadap skenario CRUD (Create, Read, Update, Delete), termasuk skenario gagal seperti data tidak lengkap, tipe data tidak sesuai, dan ID rute bus yang tidak ditemukan.

Berdasarkan hasil pengujian pada Gambar 5.1, diperoleh sebanyak 24 pengujian (*All tests*) dengan 24 pengujian berhasil (*Passed*) dan 0 pengujian gagal (*Failed*). Selain itu, tidak ditemukan error pada laporan pengujian (*Errors: 0*). Hasil tersebut menunjukkan bahwa seluruh skenario pengujian yang terdapat dalam Collection telah berhasil melewati pemeriksaan yang ditetapkan.

Dengan demikian, RESTful API Rute Bus telah memenuhi skenario pengujian yang dibuat pada Postman, baik untuk proses pengelolaan data maupun penanganan permintaan yang tidak valid.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/Zamn48/Tugas-2-Automation-Postman
Lokasi berkas Collection (.json)	C:\Users\User\Downloads\Tugas2_PAW_Zam\Postman Automation\tugas2-collection.json
Lokasi berkas Environment (.json)	C:\Users\User\Downloads\Tugas2_PAW_Zam\Postman Automation\development.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[ ] Ya [ ] Tidak
Repositori dapat dibuka dosen	[ ] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:
Menurut saya, assertion yang memeriksa status code dan isi respons sangat berguna karena keduanya membantu memastikan API memberikan hasil sesuai dengan permintaan. Pada API Rute Bus, pemeriksaan struktur data dan nilai atribut membantu mengetahui apakah data rute yang diterima sudah sesuai dengan kebutuhan pengujian

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:
Menurut saya, pengujian manual memudahkan pemeriksaan satu request secara langsung, sedangkan Collection Runner membantu menjalankan beberapa request beserta assertion secara berurutan. Bagian yang cukup menantang adalah mengelola ID rute bus dari hasil POST agar dapat digunakan pada request PUT dan DELETE. Saya mengatasinya dengan menyimpan ID ke variabel dan memastikan urutan request sesuai dengan alur pengujian.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	
2	Setiap endpoint API saya ada di Collection	
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	
4	Setiap request punya assertion status code dan struktur JSON	
5	Nama setiap test berupa kalimat yang menyatakan harapan	
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	
7	Server dinyalakan ulang sebelum Runner dijalankan	
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	
9	Berkas Environment tidak memuat kunci rahasia sungguhan	
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen